Ассистент Порфирий

Исправление 2: один заход

Новый контроллер чата и проверка, которая сама пишет вывод

Что меняется

- Контроллер больше не прерывает ответ, когда PHP решает, что браузер ушёл (`connection_aborted()`). Это то же, что делала ваша правка `sed` из диагностики 1, — новый файл её заменяет.
- На каждый ответ сервер пишет одну строку в `storage/logs/porfiry.log`: чем кончился ответ, сколько кусков ушло в браузер, видит ли PHP обрыв связи и скорость модели. Текст вопросов не пишется.
- Ответ заканчивается строкой `{"done":true}` — так проверка отличает «дошёл целиком» от «оборвался». Файл `chat.js` менять не нужно.

> Пока идёт проверка, кнопка «Стоп» и закрытая вкладка не останавливают ответ на сервере — он дописывается до `LLM_NUM_PREDICT` токенов. Остановку вернём, когда станет ясна причина обрыва.

Файлы создаются так: команда, Enter, вставить весь текст из рамки, Enter, Ctrl+D. Если `sudo` спросит пароль — сначала пароль, потом текст.

Шаг 1. Путь к сайту и копия контроллера

```
cd /путь/к/сайту
SITE=$PWD
cp app/Http/Controllers/ChatController.php ~/ChatController.php.bak-2
```

Вместо `/путь/к/сайту` — папка, где лежит `artisan`.

Шаг 2. Новый контроллер

Команда, Enter, затем весь текст контроллера (от `<?php` до последней `}`), Enter, Ctrl+D:

```
sudo tee "$SITE"/app/Http/Controllers/ChatController.php > /dev/null
```

```php
<?php

namespace App\Http\Controllers;

use Illuminate\Contracts\Cache\Lock;
use Illuminate\Http\JsonResponse;
use Illuminate\Http\Request;
use Illuminate\Support\Facades\Cache;
use Illuminate\Support\Facades\Http;
use Illuminate\Validation\ValidationException;
use Psr\Http\Message\StreamInterface;
use Symfony\Component\HttpFoundation\StreamedResponse;
use Throwable;

class ChatController extends Controller
{
    // Предел длины одного сообщения истории (ответы с кодом бывают длинными).
    private const MAX_HISTORY_MESSAGE_CHARS = 20000;

    // Сколько сообщений браузер может прислать; лишнее сервер всё равно обрежет.
    private const MAX_REQUEST_MESSAGES = 100;

    public function index()
    {
        // В браузер уходят название, лимит и адрес; роли (system) — на сервере.
        $chats = [];
        foreach (config('llm.chats') as $key => $chat) {
            $chats[$key] = [
                'title' => $chat['title'],
                'maxInput' => $chat['max_input'],
                'url' => route('chat.send', $key),
            ];
        }

        return view('chat', [
            'chats' => $chats,
            'historyLimit' => config('llm.history.max_messages'),
        ]);
    }

    public function send(
        Request $request,
        string $type
    ): JsonResponse|StreamedResponse {
        $chats = config('llm.chats');
        abort_unless(isset($chats[$type]), 404);
        $chat = $chats[$type];

        $messages = $this->validatedMessages($request, $chat['max_input']);
        $messages = $this->trimHistory($messages);

        $userLock = Cache::lock(
            'llm:session:' . $request->session()->getId(),
            config('llm.queue.lock_seconds')
        );
        if (! $userLock->get()) {
            return $this->fail('Дождитесь ответа на предыдущий вопрос.', 429);
        }

        $slotLock = $this->acquireSlot();
        if ($slotLock === null) {
            $userLock->release();

            return $this->fail('Сервер занят, попробуйте через минуту.', 503);
        }
```

```
        $release = function () use ($userLock, $slotLock) {
            $slotLock->release();
            $userLock->release();
        };

        set_time_limit(config('llm.response_timeout') + 30);

        try {
            $body = $this->openOllamaStream($chat, $messages);
        } catch (Throwable $e) {
            $release();
            $this->trace("{$type}: Ollama недоступна: " . $e->getMessage());

            return $this->fail(
                'Модель сейчас недоступна. '
                . 'Попробуйте позже или сообщите администратору.',
                503
            );
        }

        return response()->stream(
            fn () => $this->relay($body, $release, $type),
            200,
            [
                'Content-Type' => 'application/x-ndjson; charset=UTF-8',
                'Cache-Control' => 'no-cache',
                'X-Accel-Buffering' => 'no',
            ]
        );
    }

    /** @return array<int, array{role: string, content: string}> */
    private function validatedMessages(Request $request, int $maxInput): array
    {
        $max = self::MAX_REQUEST_MESSAGES;
        $data = $request->validate([
            'messages' => ['required', 'array', 'min:1', 'max:' . $max],
            'messages.*.role' => ['required', 'string', 'in:user,assistant'],
            'messages.*.content' => [
                'required', 'string', 'max:' . self::MAX_HISTORY_MESSAGE_CHARS,
            ],
        ]);

        $messages = array_map(
            fn (array $m) => ['role' => $m['role'], 'content' => $m['content']],
            array_values($data['messages'])
        );

        $last = end($messages);
        if ($last['role'] !== 'user') {
            throw ValidationException::withMessages([
                'messages' => 'Последнее сообщение должно быть '
                    . 'вопросом пользователя.',
            ]);
        }
        if (mb_strlen($last['content']) > $maxInput) {
            throw ValidationException::withMessages([
                'messages' => "Вопрос длиннее {$maxInput} символов.",
            ]);
        }

        return $messages;
    }

    /** Оставляет последние сообщения в пределах лимитов; вопрос остаётся всегда. */
```

```
    private function trimHistory(array $messages): array
    {
        $kept = array_slice($messages, -config('llm.history.max_messages'));
        $maxChars = config('llm.history.max_chars');

        while (count($kept) > 1 && $this->totalChars($kept) > $maxChars) {
            $kept = array_slice($kept, 1);
        }

        return $kept;
    }

    private function totalChars(array $messages): int
    {
        return array_sum(array_map(fn ($m) => mb_strlen($m['content']), $messages));
    }

    private function acquireSlot(): ?Lock
    {
        $seconds = config('llm.queue.lock_seconds');

        for ($i = 0; $i < config('llm.queue.slots'); $i++) {
            $lock = Cache::lock("llm:slot:{$i}", $seconds);
            if ($lock->get()) {
                return $lock;
            }
        }

        return null;
    }

    private function openOllamaStream(array $chat, array $messages): StreamInterface
    {
        $url = rtrim(config('llm.ollama_url'), '/') . '/api/chat';
        $system = ['role' => 'system', 'content' => $chat['system']];

        $response = Http::connectTimeout(config('llm.connect_timeout'))
            ->timeout(config('llm.response_timeout'))
            ->withOptions(['stream' => true])
            ->post($url, [
                'model' => $chat['model'],
                'stream' => true,
                'messages' => [$system, ...$messages],
                'options' => [
                    'temperature' => $chat['temperature'],
                    'num_ctx' => $chat['num_ctx'],
                    'num_predict' => $chat['num_predict'],
                ],
            ]);

        if ($response->failed()) {
            $text = mb_substr($response->body(), 0, 500);
            throw new \RuntimeException("HTTP {$response->status()}: {$text}");
        }

        return $response->toPsrResponse()->getBody();
    }

    /**
     * Пересылает NDJSON Ollama в браузер строками {"t": "..."},
     * в конце — {"done": true} или {"error": "..."}.
     */
    private function relay(
        StreamInterface $body,
        callable $release,
        string $type
```

```
    ): void {
        // Вкладку закрыли — дорабатываем сами, чтобы снять блокировки.
        ignore_user_abort(true);
        $buffer = '';
        $pieces = 0;
        $lostAt = 0;
        $started = microtime(true);
        $reason = 'поток Ollama кончился без признака done';

        try {
            while (! $body->eof()) {
                // По одному байту: на read(8192) http-обёртка PHP ждёт полные
                // 8 КБ, и текст шёл бы в браузер рывками, а не по словам.
                $buffer .= $body->read(1);

                while (($pos = strpos($buffer, "\n")) !== false) {
                    $line = trim(substr($buffer, 0, $pos));
                    $buffer = substr($buffer, $pos + 1);
                    if ($line === '') {
                        continue;
                    }

                    $chunk = json_decode($line, true);
                    if (isset($chunk['error'])) {
                        $reason = 'ошибка Ollama: ' . $chunk['error'];
                        $this->emit([
                            'error' => 'Модель вернула ошибку. Попробуйте ещё раз.',
                        ]);

                        return;
                    }
                    if (($chunk['message']['content'] ?? '') !== '') {
                        $this->emit(['t' => $chunk['message']['content']]);
                        $pieces++;
                        // Обрыв связи с браузером только отмечаем, ответ не
                        // прерываем: так в журнале видно, доходят ли куски.
                        // Брошенный ответ доработает до LLM_NUM_PREDICT токенов.
                        if ($lostAt === 0 && connection_aborted()) {
                            $lostAt = $pieces;
                        }
                    }
                    if (! empty($chunk['done'])) {
                        $reason = 'done (' . ($chunk['done_reason'] ?? '?') . ')'
                            . $this->speed($chunk);
                        $this->emit(['done' => true]);

                        return;
                    }
                }
            }
        } catch (Throwable $e) {
            $reason = 'исключение: ' . $e->getMessage();
            $this->emit(['error' => 'Ответ прервался. Попробуйте ещё раз.']);
        } finally {
            // Закрытие соединения с Ollama останавливает генерацию.
            $body->close();
            $release();
            $link = 'связь цела';
            if ($lostAt > 0) {
                $link = "PHP видит обрыв связи после куска {$lostAt}";
            }
            $sec = microtime(true) - $started;
            $this->trace(sprintf(
                '%s: %s; кусков %d; %s; %.1f с',
                $type, $reason, $pieces, $link, $sec
            ));
        }
```

```
    }

    /** Скорость Ollama из последней строки потока, токенов в секунду. */
    private function speed(array $chunk): string
    {
        $rate = function (string $count, string $time) use ($chunk): string {
            $n = $chunk[$count] ?? 0;
            $ns = $chunk[$time] ?? 0;

            $perSec = $ns > 0 ? $n * 1e9 / $ns : 0;

            return sprintf('%d ток. (%.1f ток/с)', $n, $perSec);
        };

        return '; вопрос ' . $rate('prompt_eval_count', 'prompt_eval_duration')
            . '; ответ ' . $rate('eval_count', 'eval_duration');
    }

    /** Чем кончился ответ — в storage/logs/porfiry.log. Текст вопросов не пишем. */
    private function trace(string $text): void
    {
        $text = preg_replace('/\s+/u', ' ', $text);
        $line = date('Y-m-d H:i:s') . ' ' . $text . "\n";
        $file = storage_path('logs/porfiry.log');
        if (@file_put_contents($file, $line, FILE_APPEND | LOCK_EX) === false) {
            error_log('PORFIRY ' . $line);
        }
    }

    private function emit(array $payload): void
    {
        echo json_encode($payload, JSON_UNESCAPED_UNICODE) . "\n";
        if (ob_get_level() > 0) {
            ob_flush();
        }
        flush();
    }

    private function fail(string $message, int $status): JsonResponse
    {
        return response()->json(['message' => $message], $status);
    }
}
```

Проверить, что вставилось целиком:

```
php -l "$SITE"/app/Http/Controllers/ChatController.php
```

Должно быть `No syntax errors detected`. Иначе повторите шаг 2.

Шаг 3. Права, кэши, перезапуск

Перезапуск Ollama заодно снимает ответы, которые ещё дописываются с прошлых попыток.

```
cd "$SITE"
sudo chown -R www-data:www-data storage bootstrap/cache
sudo -u www-data php artisan config:cache
sudo -u www-data php artisan cache:clear
sudo systemctl restart ollama
sudo systemctl restart apache2
sudo systemctl try-restart php8.1-fpm
```

Шаг 4. Проверка

Создать файл проверки: команда, Enter, текст из рамки (от `<?php` до строки `echo 'ИТОГ: '…`), Enter, Ctrl+D:

```
cat > ~/porfiry-selfcheck.php
```

```php
<?php
// porfiry-selfcheck: Ollama напрямую, чат через Apache и журнал сайта.
// Запуск: sudo php ~/porfiry-selfcheck.php "$SITE" http://127.0.0.1
$site = rtrim($argv[1] ?? '', '/');
$base = rtrim($argv[2] ?? 'http://127.0.0.1', '/');
$question = 'Ответь одним коротким предложением: что такое атом?';
$sum = [];

function say($level, $label, $text)
{
    global $sum;
    echo "{$level}  {$label}: {$text}\n";
    if ($level === 'ПРОБЛЕМА' || $level === 'ВНИМАНИЕ') {
        $sum[] = $label;
    }
}

// POST и чтение потока NDJSON: на каждую строку зовёт $onLine.
function post($url, $body, $headers, $onLine)
{
    $ctx = stream_context_create(['http' => [
        'method' => 'POST',
        'header' => implode("\r\n", $headers),
        'content' => $body,
        'timeout' => 900,
        'ignore_errors' => true,
    ]]);
    $start = microtime(true);
    $fp = @fopen($url, 'r', false, $ctx);
    if ($fp === false) {
        return [0, error_get_last()['message'] ?? 'нет связи', 0];
    }
    preg_match('/ (\d{3})/', $http_response_header[0] ?? '', $m);
    $code = (int) ($m[1] ?? 0);
    if ($code !== 200) {
        $text = stream_get_contents($fp);
        fclose($fp);
        return [$code, $text, 0];
    }
    while (($line = fgets($fp)) !== false) {
        $data = json_decode(trim($line), true);
        if (is_array($data)) {
            $onLine($data, microtime(true) - $start);
        }
    }
    $late = stream_get_meta_data($fp)['timed_out'] ? 'ожидание вышло' : '';
    fclose($fp);
    return [200, $late, microtime(true) - $start];
}

function rate($d, $count, $time)
{
    $ns = $d[$time] ?? 0;
    return $ns > 0 ? ($d[$count] ?? 0) * 1e9 / $ns : 0;
}

function short($text)
{
    $text = json_decode($text, true)['message'] ?? $text;
    $text = preg_replace('/\s+/u', ' ', strip_tags($text));
    return preg_replace('/^(.{0,160}).*$/us', '$1', trim($text));
}

if ($site === '' || ! is_file("{$site}/artisan")) {
    say('ПРОБЛЕМА', 'ПУТЬ', "в «{$site}» нет artisan — укажите путь к сайту");
    exit(1);
}
```

```
$ctl = "{$site}/app/Http/Controllers/ChatController.php";
$ctl = (string) @file_get_contents($ctl);
if (strpos($ctl, 'porfiry.log') === false) {
    say('ПРОБЛЕМА', 'КОНТРОЛЛЕР', 'старая версия — сначала замените файл');
} else {
    say('ОК', 'КОНТРОЛЛЕР', 'новая версия на месте');
}

$env = [];
foreach (@file("{$site}/.env") ?: [] as $line) {
    if (preg_match('/^\s*(LLM_[A-Z_]+)\s*=\s*"?([^"\s]*)/', $line, $m)) {
        $env[$m[1]] = $m[2];
    }
}
$ollama = rtrim($env['LLM_OLLAMA_URL'] ?? '', '/') ?: 'http://127.0.0.1:11434';
$model = ($env['LLM_MODEL_GENERAL'] ?? '') ?: 'qwen-main';
$numCtx = (int) ($env['LLM_NUM_CTX'] ?? 0) ?: 8192;
$predict = (int) ($env['LLM_NUM_PREDICT'] ?? 0) ?: 2048;
say('ИНФО', 'НАСТРОЙКИ', "{$model}, num_ctx {$numCtx}, num_predict {$predict}");

echo "...жду ответ Ollama напрямую, до 15 минут\n";
$pieces = 0;
$first = 0;
$last = [];
$body = json_encode([
    'model' => $model,
    'stream' => true,
    'messages' => [['role' => 'user', 'content' => $question]],
    'options' => ['num_ctx' => $numCtx, 'num_predict' => 60],
], JSON_UNESCAPED_UNICODE);
$onLine = function ($d, $t) use (&$pieces, &$first, &$last) {
    if (($d['message']['content'] ?? '') !== '') {
        $pieces++;
        $first = $first ?: $t;
    }
    $last = $d;
};
[$code, $err] = post("{$ollama}/api/chat", $body, [
    'Content-Type: application/json',
], $onLine);
if ($code !== 200) {
    say('ПРОБЛЕМА', 'OLLAMA', "не отвечает ({$code}): " . short($err));
} elseif (empty($last['done'])) {
    say('ПРОБЛЕМА', 'OLLAMA', "поток оборвался, кусков {$pieces} {$err}");
} else {
    $gen = rate($last, 'eval_count', 'eval_duration');
    $read = rate($last, 'prompt_eval_count', 'prompt_eval_duration');
    $min = $predict / max($gen, 0.1) / 60;
    $text = sprintf('кусков %d, первый через %.0f с', $pieces, $first);
    say('ОК', 'OLLAMA', $text);
    $level = $gen < 2 ? 'ПРОБЛЕМА' : ($gen < 5 ? 'ВНИМАНИЕ' : 'ОК');
    say($level, 'СКОРОСТЬ', sprintf(
        'пишет %.1f ток/с, читает %.1f ток/с; ответ в %d ток. ≈ %.0f мин',
        $gen, $read, $predict, $min
    ));
}

$ctx = ['http' => ['timeout' => 60, 'ignore_errors' => true]];
$ctx = stream_context_create($ctx);
$page = @file_get_contents("{$base}/chat", false, $ctx);
$cookies = [];
foreach ($http_response_header ?? [] as $h) {
    if (preg_match('/^Set-Cookie:\s*([^=;]+=[^;]*)/i', $h, $m)) {
        $cookies[] = $m[1];
    }
}
preg_match('/name="csrf-token" content="([^"]+)"/', (string) $page, $m);
$token = $m[1] ?? '';
```

```
$sPieces = 0;
$sFirst = 0;
$done = false;
$sErr = '';
$started = time();
if ($token === '') {
    say('ПРОБЛЕМА', 'САЙТ', "{$base}/chat не открылась или нет CSRF-токена");
} else {
    echo "...жду ответ чата через Apache, до 15 минут\n";
    $body = json_encode([
        'messages' => [['role' => 'user', 'content' => $question]],
    ], JSON_UNESCAPED_UNICODE);
    $onLine = function ($d, $t) use (&$sPieces, &$sFirst, &$done, &$sErr) {
        if (isset($d['t'])) {
            $sPieces++;
            $sFirst = $sFirst ?: $t;
        }
        $done = $done || ! empty($d['done']);
        $sErr = $d['error'] ?? $sErr;
    };
    [$code, $err, $total] = post("{$base}/chat/general", $body, [
        'Content-Type: application/json',
        'Accept: application/json',
        'X-CSRF-TOKEN: ' . $token,
        'Cookie: ' . implode('; ', $cookies),
    ], $onLine);
    if ($code !== 200) {
        say('ПРОБЛЕМА', 'ЧАТ', "сайт ответил {$code}: " . short($err));
    } elseif ($done && $sPieces > 1) {
        say('ОК', 'ЧАТ', sprintf(
            'ответ дошёл целиком: кусков %d, первый через %.0f с, всего %.0f с',
            $sPieces, $sFirst, $total
        ));
    } else {
        $why = ", без конца ответа {$err}";
        if ($sErr !== '') {
            $why = ", ошибка «{$sErr}»";
        }
        say('ПРОБЛЕМА', 'ЧАТ', "ответ оборвался: кусков {$sPieces}{$why}");
    }
}

$log = "{$site}/storage/logs/porfiry.log";
$trace = '';
foreach (@file($log) ?: [] as $line) {
    $trace = trim($line);
}
clearstatcache();
$fresh = $trace !== '' && filemtime($log) >= $started - 2;
foreach (['/var/log/apache2/error.log', '/var/log/php8.1-fpm.log'] as $f) {
    foreach ($fresh ? [] : (@file($f) ?: []) as $line) {
        if (($at = strpos($line, 'PORFIRY ')) !== false) {
            $trace = trim(substr($line, $at + 8)) . ' (журнал веб-сервера)';
        }
    }
}
say('ИНФО', 'ЖУРНАЛ', $trace ?: 'нет записей');

if ($token !== '' && ! ($done && $sPieces > 1)) {
    $sent = preg_match('/кусков (\d+)/u', $trace, $m) ? (int) $m[1] : 0;
    if (! $fresh) {
        $cause = 'PHP не дописал журнал: смотрите error.log Apache и PHP-FPM';
    } elseif (strpos($trace, 'обрыв связи') !== false || $sent > $sPieces) {
        $cause = "PHP отправил кусков {$sent}, дошло {$sPieces}: рвёт Apache";
    } elseif (strpos($trace, 'done (length)') !== false) {
        $cause = 'модель упёрлась в лимит длины: LLM_NUM_PREDICT или num_ctx';
    } elseif (strpos($trace, 'без признака done') !== false) {
        $cause = 'PHP перестал получать поток от Ollama';
```

```
    } else {
        $cause = 'смотрите строку ЖУРНАЛ выше';
    }
    say('ПРОБЛЕМА', 'ПРИЧИНА', $cause);
}
echo 'ИТОГ: ' . ($sum ? implode(' ', $sum) : 'чат работает') . "\n";
```

Запустить. Второе — адрес сайта с самого сервера, как в `check-site.py --base`:

```
sudo php ~/porfiry-selfcheck.php "$SITE" http://127.0.0.1
```

Проверка спрашивает модель дважды: напрямую и через сайт. На медленной машине это 3–10 минут — дождитесь строки `ИТОГ:`.

Сразу после этого задайте в чате в браузере тот же вопрос: «Ответь одним коротким предложением: что такое атом?». Когда ответ закончится, посмотрите последнюю строку журнала:

```
tail -1 "$SITE"/storage/logs/porfiry.log
```

Что значит результат

Что видно	Что это значит и что делать
`ИТОГ: чат работает`, в браузере ответ целиком	Готово. Если медленно — раздел «Скорость».
`ИТОГ: чат работает`, а в браузере одна буква	Сервер исправен — ответ режет что-то между сервером и вашим ПК: прокси или антивирус с проверкой веб-трафика. Добавьте адрес сервера в их исключения. В строке `tail -1` после вопроса из браузера будет «PHP видит обрыв связи после куска 1».
`ПРИЧИНА: … рвёт Apache`	PHP отправляет ответ, но Apache его не доносит. Нужен следующий шаг — см. последний раздел.
`ПРИЧИНА: PHP перестал получать поток от Ollama`	Ollama оборвала ответ. Сфотографируйте вывод `sudo journalctl -u ollama -n 30 --no-pager`.
`ПРИЧИНА: модель упёрлась в лимит длины`	В `.env` проверьте `LLM_NUM_PREDICT=1000` и `LLM_NUM_CTX=4096`, затем шаг 3.
`ПРИЧИНА: PHP не дописал журнал`	PHP прервали посреди ответа. Сфотографируйте `sudo tail -20 /var/log/apache2/error.log` и `sudo tail -20 /var/log/php8.1-fpm.log`.
`OLLAMA: не отвечает`	`sudo systemctl status ollama` — должно быть `active`. Затем шаг 3.
`КОНТРОЛЛЕР: старая версия`	Шаг 2 не сработал или в `SITE` другой путь.
`САЙТ: … не открылась`	Неверный адрес во втором аргументе — укажите тот, что в `check-site.py --base`.
`ЖУРНАЛ: …` с «исключение» или «ошибка Ollama»	Перепишите текст после этих слов.

Скорость

Строка `СКОРОСТЬ` — сколько токенов в секунду модель пишет на этой машине. Проверочный файл из диагностики 2 получил 200 строк за 240 секунд — около 1 токена в секунду. Для этой модели нормально 5–10 ток/с на 4 ядрах с AVX2.

При 1 ток/с ответ в 1000 токенов идёт около 17 минут, а два параллельных ответа делят процессор. 30 человек при такой скорости ждут десятки минут — это предел машины, а не ошибка сайта.

- Короче ответы — быстрее очередь. Поставить 400 токенов, затем шаг 3:

```
sudo sed -i 's/^LLM_NUM_PREDICT=.*/LLM_NUM_PREDICT=400/' "$SITE"/.env
grep NUM_PREDICT "$SITE"/.env
```

- Главное средство — больше ядер виртуалке и тип процессора host-passthrough в настройках гипервизора: виртуалка получает все инструкции процессора хоста.
- Если ядер не добавить — модель поменьше (Qwen3-1.7B): примерно вдвое быстрее, но слабее. Её нужно принести файлом.

Тесты (по желанию)

Один тест на сервере ждёт старый формат ответа. Обновить его и прогнать тесты:

```
cd "$SITE"/tests/Feature
sudo sed -i 's/вет!\\"}\\n"/вет!\\"}\\n{\\"done\\":true}\\n"/' ChatTest.php
cd "$SITE"
sudo -u www-data php artisan config:clear
sudo -u www-data php artisan test
sudo -u www-data php artisan config:cache
```

Откат

```
sudo cp ~/ChatController.php.bak-2 "$SITE"/app/Http/Controllers/ChatController.php
sudo systemctl restart apache2
sudo systemctl try-restart php8.1-fpm
```

Что сообщить разработчику

- Вывод проверки: строку `ИТОГ`, строки `ПРОБЛЕМА` и `ВНИМАНИЕ`, строки `СКОРОСТЬ` и `ЖУРНАЛ`.
- Что показал браузер и строку `tail -1` после вопроса из браузера.
- Только если `ПРИЧИНА: … рвёт Apache` — вывод двух команд:

```
cd /etc/apache2
sudo grep -rn "fcgi\|flush\|Timeout\|deflate" sites-enabled conf-enabled
sudo tail -20 /var/log/apache2/error.log
```